

Dynamic Arrays

An Introduction

Department of Computer Science
University of Pretoria

28 May 2024

Write a matrix addition function,

```
void addMatrices(float a[][4], float b[][4], float result[][4], int  
rows, int cols);
```

Matrix addition is defined as follows: For two matrices A and B of the same dimensions $m \times n$:

$$C_{ij} = A_{ij} + B_{ij}, 0 \leq i < m, 0 \leq j < n$$

where C is the resulting matrix.

What do we know?

- Both matrices are $m \times n$
- The resulting matrix is also $m \times n$
- Consider the following example:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} + \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 1+1 & 0+0 & 0+1 & 0+0 \\ 0+0 & 1+1 & 1+0 & 0+1 \\ 1+1 & 0+0 & 0+1 & 1+0 \end{bmatrix}$$
$$= \begin{bmatrix} 2 & 0 & 1 & 0 \\ 0 & 2 & 1 & 1 \\ 2 & 0 & 1 & 1 \end{bmatrix}$$

Now write the function:

```
void addMatrices(float a[][4], float b[][4], float result[][4], int  
rows, int cols);
```

- Require 2 loops, one to iterate per row (i) and one per column (j)
- i runs from 0 to rows - 1
- j runs from 0 to cols - 1
- $\text{result}[i][j] = \text{a}[i][j] + \text{b}[i][j]$

Write `void addMatrices(float a[][4], float b[][4], float result[][4], int rows, int cols);`

- Require 2 loops, one to iterate per row (i) and one per column (j)
- i runs from 0 to rows - 1
- j runs from 0 to cols - 1
- `result[i][j] = a[i][j] + b[i][j]`

```
void addMatrices(float a[][4], float b[][4], float result[][4],
                int rows, int cols) {
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            result[i][j] = a[i][j] + b[i][j];
        }
    }
}
```

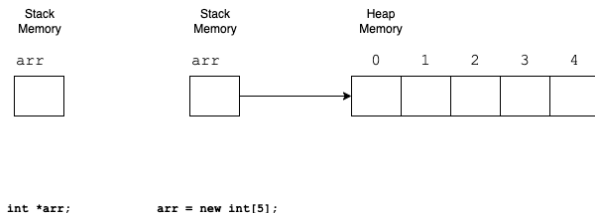
- Our matrices may have less than 4 rows, but must have 4 columns. This is due to the requirement that the second array index must be a constant (4) when sending a static array as a parameter to the functions we write.
- How can we generalise this to make provision for multiple size matrices and especially when we need to multiply matrices?
- Matrix multiplication is defined as:
For matrices A of dimension $m \times n$ and B of dimension $n \times p$:

$$C_{ij} = \sum_{k=0}^{n-1} A_{ik} \times B_{kj}, 0 \leq i < m, 0 \leq j < p$$

where C is the resulting matrix of dimension $m \times p$.

- In this case, if $n = 4$ and $p = 4$ due to the parameters we have been passing, then matrix B must be 4×4
- So far, we have known the size of the array at compile-time. We require some way of creating arrays more dynamically.
- We want to define the number of rows and columns an array has at run-time.
- This means we need to make use of heap memory and create the array using `new`. It will mean that we need to take responsibility to delete the memory.

- To create a 1D dynamic array, we require a pointer to the first element of the array. For example: `int *arr;`
- We can then reserve the contiguous memory on the heap for the array based on the size we require, `arr = new int[5];`
- We take responsibility to delete the array, `delete [] arr;`



```
#include <iostream>
using namespace std;

int main() {
    int *arr;
    int size;

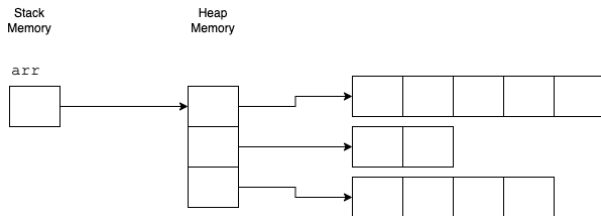
    cout << "How many elements would you like to store? ";
    cin >> size;
    arr = new int[size];

    cout << "Please enter your " << size << " values" << endl;
    for (int i = 0; i < size; i++) {
        cout << i + 1 << ": "; cin >> arr[i];
    }

    cout << "The values entered are: ";
    for (int i = 0; i < size; i++) { cout << arr[i] << " "; }
    cout << endl;

    delete [] arr;
    return 0;
}
```

- 2D dynamic arrays become a little more complex.
- We typically place a 1D array on the heap representing the rows.
- Each element in the the 1D array is a pointer to another 1D array
- Each row of a 2D dynamic array does not need to have the same number of “columns”



- These structures become difficult to manage. It is necessary to keep track of how many “columns” are linked to each “row”

Now create a 2×3 matrix using 2D dynamic arrays

- We make use of double pointers to define the initial variable on the stack:

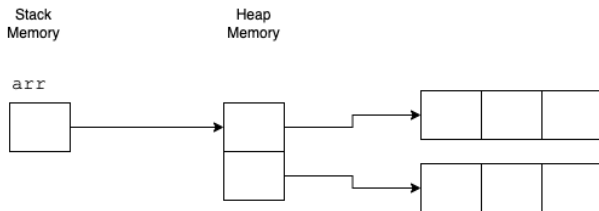
```
float **arr;  
int rows = 2, cols = 3;
```

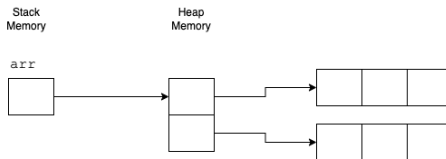
- Create the array representing the 2 rows:

```
arr = new float*[rows];
```

- Create 2 arrays with 3 elements and link them to the rows:

```
for (int i = 0; i < rows; i++) {  
    arr[i] = new float[cols];  
}
```





- Read values into the arrays

```
cout << "Type in the matrix:" << endl;
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        cout << "[" << i << "]"[" << j << "] ";
        cin >> arr[i][j];
    }
}
```

- Delete the matrix

```
for (int i = 0; i < rows; i++)
    delete [] arr[i];
delete [] arr;
arr = NULL;
```

- Write the code to add two matrices
- Previously the function was defined as:

```
void addMatrices(float a[][4], float b[][4], float result[][4],  
                int rows, int cols);
```

- We can now define the function as follows:

```
float** addMatrices(float **a, float **b, int rows, int cols);
```

- To help manage the matrices, write functions to allocate the memory on the heap and free the memory from the heap representing the matrices. You can add these functions to your matrix library.

```
float** allocateMatrix(int rows, int cols) {  
    float** matrix = new float*[rows];  
    for (int i = 0; i < rows; ++i)  
        matrix[i] = new float[cols];  
    return matrix;  
}  
  
void deallocateMatrix(float** matrix, int rows) {  
    for (int i = 0; i < rows; ++i)  
        delete[] matrix[i];  
    delete[] matrix;  
    matrix = NULL;  
}
```

- A printMatrix function is always handy ;-)

```
void printMatrix(float** matrix, int rows, int cols);
```

- Write the addMatrices function

```
float** addMatrices(float **a, float **b, int rows, int cols) {  
    float** matrix = allocateMatrix(rows, cols);  
  
    for (int i = 0; i < rows; ++i) {  
        for (int j = 0; j < cols; ++j) {  
            matrix[i][j] = a[i][j] + b[i][j];  
        }  
    }  
  
    return matrix;  
}
```

- What about the multiplyMatrices function? This stands over again for homework.
Remember the definition is given by: For matrices A of dimension $m \times n$ and B of dimension $n \times p$:

$$C_{ij} = \sum_{k=0}^{n-1} A_{ik} \times B_{kj}, 0 \leq i < m, 0 \leq j < p$$

where C is the resulting matrix of dimension $m \times p$.